Understanding Recurrences and Master Theorem in Algorithm Analysis

Recurrences are equations or inequalities that describe the running time of a recursive algorithm. They are used to characterize the runtime in terms of the size of the input. Recurrences express the runtime $T(n)$ of a problem of size n in terms of the runtime T on smaller inputs.

General Form of Recurrences

A common form of recurrence relations is:

$$ T(n) = aT\left(\frac{n}{b}\right) + f(n) $$

- **a**: The number of subproblems in the recursion.
- **b**: The factor by which the subproblem size is divided.
- **$f(n)$**: The cost outside the recursive calls, often referred to as the "combining" cost.

Master Theorem for Divide-and-Conquer Recurrences

The **Master Theorem** provides a straightforward way to solve recurrences of the form:

$$ T(n) = aT\left(\frac{n}{b}\right) + f(n) $$

Cases of Master Theorem

The Master Theorem applies to divide-and-conquer recurrences and has three cases:

1. **Case 1: $ f(n) = O(n^c) $ where $ c < \log_b{a} $**:
   - Solution: $ T(n) = O(n^{\log_b{a}}) $

2. **Case 2: $f(n) = O(n^c)$ where $c = \log_b{a}$**:
   - Solution: $T(n) = O(n^c \log{n})$ (or equivalently $T(n) = O(n^{\log_b{a}} \log{n})$)

3. **Case 3: $f(n) = O(n^c)$ where $c > \log_b{a}$**:
   - Solution: $T(n) = O(f(n))$

Applying the Master Theorem

To use the Master Theorem, identify a, b, and $f(n)$ in your recurrence relation and compare $f(n)$ with $n^{\log_b{a}}$.

Examples of Recurrence Relations

1. **Merge Sort**:
   $$ T(n) = 2T\left(\frac{n}{2}\right) + O(n) $$
   - Here, $a = 2$, $b = 2$, and $f(n) = O(n)$.
   - $\log_b{a} = \log_2{2} = 1$
   - Since $f(n) = O(n)$ and $n = n^{\log_2{2}}$, it falls into Case 2.
   - Solution: $T(n) = O(n \log n)$

2. **Binary Search**:
   $$ T(n) = T\left(\frac{n}{2}\right) + O(1) $$
   - Here, $a = 1$, $b = 2$, and $f(n) = O(1)$.
   - $\log_b{a} = \log_2{1} = 0$
   - Since $f(n) = O(1)$ and $1 = n^0$, it falls into Case 2.
   - Solution: $T(n) = O(\log n)$

3. **Strassen's Matrix Multiplication**:

\[T(n) = 7T\left(\frac{n}{2}\right) + O(n^2) \]

- Here, \(a = 7\), \(b = 2\), and \(f(n) = O(n^2)\).

- \(\log_b{a} = \log_2{7} \approx 2.81 \)

- Since \(f(n) = O(n^2) \) and \(n^2 < n^{\log_2{7}} \), it falls into Case 1.

- Solution: \(T(n) = O(n^{\log_2{7}}) \)

Recurrences and Algorithm Run Times

Recurrences are crucial in describing the run times of recursive algorithms because they allow you to break down the problem into smaller subproblems, analyze the cost of each subproblem, and then combine these costs to get the total runtime.

- **Divide-and-Conquer Algorithms**: Such as merge sort and quicksort, naturally lend themselves to recurrence relations because they break the problem into subproblems of smaller sizes, solve the subproblems recursively, and then combine the results.

- **Dynamic Programming Algorithms**: Often analyzed using recurrences to determine the cost of solving subproblems and combining them to solve larger problems.

By solving these recurrences, either through the Master Theorem or other methods (such as substitution or recursion-tree method), you can accurately determine the time complexity of an algorithm.

Understanding and applying the Master Theorem is a powerful tool in algorithm analysis, helping you quickly solve complex recurrence relations and providing insights into the efficiency of algorithms.